

Relatórios Ex7

Exercício 1 — Figura complexa usando linhas

1. Objetivo do Programa

O código é uma aplicação interativa de Computação Gráfica 2D. O objetivo principal é criar uma janela onde o usuário, ao clicar com o botão esquerdo do mouse, define a posição para desenhar a figura geométrica de um foguete. A aplicação utiliza as bibliotecas GLFW para o gerenciamento da janela e captura de eventos, e PyOpenGL (OpenGL.GL) para a renderização gráfica.

2. Dependências

Para que o código funcione corretamente, é necessário ter o Python instalado com as seguintes bibliotecas:

- **glfw**: Gerencia a criação da janela, o contexto OpenGL e a entrada de periféricos (mouse, teclado).
- **PyOpenGL**: Fornece os bindings do Python para a API gráfica do OpenGL.

3. Estrutura e Componentes Principais

O código está estruturado de forma procedural e pode ser dividido nos seguintes componentes lógicos:

3.1. Gerenciamento de Estado e Sistema de Coordenadas

- **Variáveis Globais**: `posicao_clique`, `largura_janela` e `altura_janela` mantêm o estado atual da aplicação.
- **`converter_coordenadas(x_tela, y_tela)`**: É uma função matemática crucial. As coordenadas de tela (pixels) começam com (0,0) no canto superior esquerdo e vão até a largura/altura da janela. O OpenGL, no entanto, utiliza um sistema de Coordenadas de Dispositivo Normalizadas (NDC), onde o centro é (0,0) e os limites vão de -1.0 a 1.0 em ambos os eixos. Esta função mapeia o clique em pixels para o plano cartesiano do OpenGL e inverte o eixo Y.

3.2. Tratamento de Eventos (Callbacks)

- **evento_clique_mouse(...)**: Monitora a interação do usuário. Se o botão esquerdo do mouse for pressionado (`glfw.MOUSE_BUTTON_LEFT` e `glfw.PRESS`), ele captura a posição (x, y) do cursor, converte para o formato OpenGL e atualiza a variável global `posicao_clique`.
- **evento_redimensionar_janela(...)**: Garante que o desenho não fique distorcido ou seja cortado se o usuário alterar o tamanho da janela. Ele atualiza as variáveis de dimensão e ajusta a área de visualização do OpenGL usando `glViewport`.

3.3. Renderização Gráfica (Desenho do Foguete)

A função **desenhar_foguete(cx, cy)** recebe o ponto central do clique e desenha o foguete usando diferentes primitivas gráficas do OpenGL antigo (Immediate Mode):

- **Corpo (Branco)**: Utiliza `GL_LINE_LOOP` para desenhar os 4 lados de um retângulo de forma contínua, ligando o último ponto ao primeiro.
- **Bico (Vermelho)**: Utiliza `GL_LINE_STRIP` com 3 vértices para desenhar um formato de triângulo aberto sobreposto ao topo do corpo.
- **Asas e Fogo (Azul e Laranja)**: Utiliza `GL_LINES`. Essa primitiva trata cada par de vértices como uma linha independente. É ideal para desenhar os detalhes separados do foguete (as aletas laterais e os rastros de propulsão).
- *Detalhe visual*: A função aumenta a espessura das linhas com `glLineWidth(2.0)` e depois as restaura para 1.0 ao final.

3.4. Loop Principal e Renderização da Cena

- **desenhar_cena()**: Limpa o buffer de cor da tela em cada quadro e verifica se a variável `posicao_clique` possui algum valor. Se sim, invoca o desenho do foguete nas coordenadas registradas.
- **main()**: A espinha dorsal do script. Inicializa o GLFW, cria a janela (800x600), registra os callbacks, define a cor de fundo (azul escuro para simular o espaço) e entra no loop contínuo de renderização (`while not glfw.window_should_close`). Em cada iteração, ele processa os eventos do mouse/teclado, desenha a cena e troca os buffers (`swap_buffers`) para exibir o novo quadro na tela.

4. Resumo da Execução

1. O programa abre uma janela com fundo azul escuro.
2. O sistema entra em um loop infinito aguardando ações do usuário.
3. Quando o usuário clica com o botão esquerdo do mouse, a posição do ponteiro em pixels é convertida para coordenadas cartesianas matemáticas.

4. No próximo ciclo de atualização de tela (fração de segundo depois), o OpenGL renderiza um foguete de linhas coloridas (branco, vermelho, azul e laranja) exatamente no local em que o usuário clicou.
5. O foguete "se teletransporta" para novos locais a cada novo clique, pois o código não mantém um histórico de posições, mas sim atualiza uma única `posicao_clique`.

5. Conclusão

O código é um excelente exemplo didático introdutório aos fundamentos. Ele demonstra com clareza o uso de sistemas de coordenadas virtuais, primitivas básicas de desenho bidimensional (`GL_LINE_LOOP`, `GL_LINE_STRIP`, `GL_LINES`), controle de estado (cores e espessuras) e mapeamento de interatividade do usuário via bibliotecas de janela.

Exercício 2 - Triângulo Construído com Três Cliques

1. Objetivo do Programa

O código consiste em uma aplicação interativa de Computação Gráfica 2D focada no controle de estados e persistência de dados em cena. O objetivo principal é permitir que o usuário construa múltiplos triângulos na tela de maneira dinâmica, onde cada figura é gerada de forma personalizada a partir de três cliques sucessivos com o botão esquerdo do mouse. A aplicação utiliza a biblioteca GLFW para gerenciar a janela e os inputs, e a PyOpenGL (OpenGL.GL) para a renderização geométrica.

2. Dependências

Para que o código funcione corretamente, é necessário ter o Python instalado com as seguintes bibliotecas:

- **glfw:** Gerencia a criação da janela, o contexto OpenGL e a captura de eventos de entrada (cliques do mouse e teclas pressionadas).
- **PyOpenGL:** Fornece as diretivas e bindings do Python para a API gráfica do OpenGL.

3. Estrutura e Componentes Principais

O código segue um modelo procedural estruturado, organizando as responsabilidades de armazenamento, conversão e desenho nos seguintes componentes lógicos:

3.1. Gerenciamento de Estado e Sistema de Coordenadas

- **Variáveis Globais:** As listas `triangulos_completos` e `pontos_temporarios`, junto com `largura_janela` e `altura_janela`, mantêm a memória da cena.
- **`converter_coordenadas(x_tela, y_tela)`:** Executa a conversão matemática linear das coordenadas do clique em pixels (origem no topo esquerdo) para o plano de Coordenadas de Dispositivo Normalizadas (NDC) do OpenGL (escala de -1.0 a 1.0 com o eixo Y invertido).

3.2. Tratamento de Eventos (Callbacks)

- **`evento_clique_mouse(...)`:** Monitora os cliques do usuário. Quando o botão esquerdo é pressionado (`glfw.PRESS`), a posição do cursor é obtida, convertida e adicionada à lista `pontos_temporarios`. Ao acumular o 3º ponto, a lógica transfere esses vértices como uma lista definitiva para dentro de `triangulos_completos` e limpa o buffer temporário, reiniciando o ciclo para o próximo polígono.
- **`evento_teclado(...)`:** Captura as interações com o teclado. Se o usuário pressionar a tecla 'C' (`glfw.KEY_C`), a função aciona um reset de estado limpando ambas as listas de armazenamento, o que limpa a tela imediatamente.
- **`evento_redimensionar_janela(...)`:** Atualiza as dimensões globais da janela em tempo real e reajusta a área de projeção com `glViewport` para evitar distorções nos triângulos caso a janela mude de tamanho.

3.3. Renderização Gráfica e Feedback Visual

Diferente do Exercício 1, a função `desenhar_cena()` acumula a responsabilidade de interpretar a máquina de estados para renderizar múltiplos elementos com diferentes primitivas:

- **Triângulos Concluídos (Azul e Branco):** Percorre a lista histórica. Utiliza `GL_TRIANGLES` com preenchimento azul claro para renderizar as faces sólidas. Logo em seguida, altera o modo de polígono com `glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)` para desenhar um contorno branco sobre os triângulos, garantindo boa distinção visual entre formas sobrepostas.
- **Vértices e Linhas Guia (Vermelho):** Caso existam cliques pendentes na lista `pontos_temporarios`, o OpenGL altera o tamanho do ponto com `glPointSize(8.0)` e os desenha com `GL_POINTS` em vermelho. Se o usuário já tiver realizado exatamente 2 cliques, a primitiva `GL_LINES` é ativada para desenhar uma linha elástica conectando os dois primeiros vértices, servindo de guia antes do clique final.

3.4. Loop Principal e Atualização da Cena

O método `main()` coordena o ciclo de vida do software. Ele inicializa o ambiente GLFW, define a janela inicializada em `800 \times 600` pixels, vincula os três métodos de callback criados e estabelece um cinza escuro como cor de limpeza de fundo (`glClearColor`). Dentro do laço `while`, o programa processa os eventos pendentes, invoca a renderização de tudo o que está armazenado nas listas através de `desenhar_cena()` e realiza a troca de buffers (`swap_buffers`) para manter a atualização estável e fluida.

4. Resumo da Execução

- O programa inicia exibindo uma janela vazia com fundo cinza escuro.
- Quando o usuário faz o **1º clique**, um ponto vermelho com espessura realçada surge na tela.
- Ao realizar o **2º clique**, um novo ponto vermelho aparece e uma linha vermelha passa a conectar os dois vértices.
- No momento em que ocorre o **3º clique**, o estado se transforma: os pontos vermelhos e a linha guia desaparecem, dando lugar a um triângulo preenchido em azul com bordas brancas bem delineadas.
- O usuário pode continuar clicando indefinidamente; cada novo trio de cliques gera uma nova forma geométrica que passa a coexistir de forma permanente com as formas anteriores.
- A qualquer momento, pressionar a tecla '**C**' remove instantaneamente todos os triângulos e pontos da tela, restaurando o estado original limpo.

5. Conclusão

O segundo exercício expande com sucesso os conceitos de interatividade ao introduzir a persistência de dados através de estruturas dinâmicas de vetores (listas de listas). A aplicação demonstra de forma prática como implementar uma máquina de estados baseada em eventos assíncronos de mouse, oferecendo um excelente exemplo de feedback visual progressivo ao usuário (através de pontos, linhas e polígonos preenchidos que se transformam de acordo com o estágio do input) e aplicando técnicas fundamentais de desenho em Immediate Mode da OpenGL.